

HireLens

AI-Powered CV Rating System for the Pakistani Job Market

Project Documentation — Revision 2

Developer	Aadil Kamal Khan
GitHub Repository	github.com/aadilkk18/HireLens
Live Frontend	hire-lens-lime.vercel.app
Live Backend API	aadilkk18-hirelens-backend.hf.space
Document Date	July 2026 (Revision 2 — supersedes the July 2026 v1 draft)

Revision Notes

This is an updated revision of the original HireLens project documentation. The core product concept, architecture, and scoring philosophy are unchanged from v1. The changes below reflect engineering decisions made during hardening and production deployment of the backend.

Changes since the previous revision

- LLM provider switched from Groq (llama-3.3-70b-versatile) to Google Gemini (gemini-3.1-flash-lite via the google-genai SDK), for model quality and JSON-following reliability.
- A brief experiment enabling Gemini's Google Search grounding tool for university/employer verification was tried and reverted after it triggered 429 RESOURCE_EXHAUSTED errors in production; grounding consumes a separate, more restrictive quota pool and adds internal round-trips per request, which made it unsuitable for the current free-tier plan.
- The backend's Gemini prompt was rewritten with explicit scoring calibration anchors, edge-case handling, and a stricter JSON-only output contract, and now receives the current date at request time so it no longer misflags recent or same-month dates as fabricated — an issue found during real-CV testing.
- Upload handling was hardened: file size caps, magic-byte content verification (not just trusting the file extension), and explicit handling for empty, corrupted, or unreadable PDFs and images, all returning structured JSON errors instead of raw server errors.
- OCR preprocessing (grayscale conversion and contrast normalization) was added before Tesseract to improve text extraction accuracy on scanned and photographed CVs.
- Tesseract and Poppler path resolution was made fully cross-platform and automatic (PATH detection first, with optional TESSERACT_CMD / POPPLER_PATH environment overrides and a Windows-only fallback default), replacing the previous hardcoded Windows paths that caused OCR to fail on the Linux deployment.
- Gemini response parsing was made more resilient to malformed output: markdown code fences are stripped and a regex-based fallback recovers the JSON object even if the model adds stray commentary around it.
- Structured logging was added across the request lifecycle (filename, upload size, OCR fallback usage, Gemini request/response, and parsing failures) without ever logging the API key, to make production issues diagnosable from Hugging Face Space logs alone.

1. Project Overview

HireLens is a full-stack, AI-powered CV screening and rating application designed specifically around how hiring actually works in the Pakistani job market. Rather than applying generic, Western-centric resume-scoring logic, HireLens evaluates a candidate's CV the way a Pakistani corporate recruiter would — accounting for local university tiers, CGPA conventions, mixed English/Urdu formatting norms, and the volume-driven screening reality of platforms such as Rozee.pk.

The system accepts a CV in PDF, PNG, or JPG format, extracts its text using a hybrid direct-extraction and OCR pipeline, matches the candidate's intended job role against a curated list of over 115 roles using semantic similarity search, and produces a structured, weighted rating across six evaluation categories along with specific, actionable feedback.

1.1 Motivation

The project began from a simple observation during the developer's own job search: generic CV-scoring tools built on Western hiring assumptions fail to capture the specific signals Pakistani recruiters actually weigh — university reputation and tiering, the presence or absence of a CGPA, mixed-language content, and the sheer volume-based triage that platforms like Rozee.pk apply, where an average of hundreds of applicants compete for a single listing and only a fraction are ever reviewed in detail. HireLens was built to close that gap with a rating engine grounded in real research into local hiring practices.

1.2 Objectives

- Build a CV analysis tool that reflects genuine Pakistani corporate hiring criteria, not generic Western ATS heuristics.

- Support the three most common CV file formats submitted by real applicants: PDF, PNG, and JPG (including scanned or photographed CVs).
- Allow rating against a specific target job role, either selected from a comprehensive predefined list or typed freely and resolved automatically.
- Weight the final score toward the factors that matter most in real hiring decisions — skills and experience — rather than cosmetic formatting.
- Deploy the finished system as a live, publicly accessible product at zero hosting cost.
- Ensure the extraction, OCR, and rating pipeline degrades gracefully — with clear JSON error responses rather than server crashes — when given malformed, corrupted, or unsupported uploads.

2. Technology Stack

2.1 Frontend

Framework	React (Vite build tooling)
Language	JavaScript (JSX)
Styling	Custom CSS with a design-token system (CSS variables)
Hosting	Vercel

2.2 Backend

Framework	FastAPI (Python 3.13)
Server	Uvicorn (ASGI)
LLM Provider	Google Gemini API (google-genai) — gemini-3.1-flash-lite
Semantic Matching	sentence-transformers (all-MiniLM-L6-v2)
PDF Text Extraction	PyMuPDF (fitz)
OCR Engine	Tesseract OCR (pytesseract), with grayscale + autocontrast preprocessing
PDF-to-Image (OCR fallback)	pdf2image (Poppler)
Hosting	Hugging Face Spaces (Docker SDK)

2.3 Supporting ML Libraries

- transformers, torch — underlying model runtime for sentence-transformers
- scikit-learn, numpy, scipy — numerical operations supporting the embedding pipeline

3. System Architecture

HireLens follows a decoupled client-server architecture, with the frontend and backend deployed independently and communicating over a REST API secured with CORS.

3.1 High-Level Flow

User

```
|
v
React Frontend (Vercel)
| -- multipart/form-data (CV file + role) -->
v
FastAPI Backend (Hugging Face Spaces, Docker)
|
|--> Upload validation (size cap, extension + magic-byte check)
|--> Text extraction (PyMuPDF, with Tesseract OCR fallback + preprocessing)
|--> Role resolution (sentence-transformer embedding similarity)
|--> LLM rating call (Gemini API, Pakistani-recruiter system prompt, date-aware)
|--> Weighted score computation (deterministic, category-weighted)
v
Structured JSON response --> rendered in Results Dashboard
```

3.2 Repository Structure

The project is split across two repositories due to a constraint of the chosen free backend hosting platform (Hugging Face Spaces deploys a single application per Space):

Main Development Repository — github.com/aadilkk18/HireLens

- frontend/ — React application (source of truth for UI)
- backend/ — FastAPI application (source of truth for API logic)
- .gitignore

Backend Deployment Repository — [Hugging Face Git \(hirelens_backend\)](#)

- Dockerfile
- requirements.txt
- main.py
- role_resolver.py

Backend code changes are made in HireLens/backend first, then copied into `hirelens_backend` and pushed to trigger a Hugging Face rebuild. Frontend changes only require a push to the main GitHub repository, which Vercel automatically redeploys.

4. Core Features

4.1 Multi-Format File Ingestion

HireLens accepts CVs in three formats and applies the appropriate extraction strategy for each:

- PDF (text-based): text extracted directly via PyMuPDF.
- PDF (scanned / image-based): automatic fallback — if direct extraction yields fewer than 30 characters, the PDF is rasterized page-by-page via `pdf2image` and passed through Tesseract OCR.
- PNG / JPG: processed directly through Tesseract OCR.

Before OCR runs on any image — whether a standalone upload or a rasterized PDF page — the image is converted to grayscale and contrast-normalized. This preprocessing step was added after testing showed it measurably improves text recognition accuracy on lower-quality scanned and phone-photographed CVs.

4.2 Upload Validation and Security

Every upload passes through a validation layer before extraction begins, to prevent crashes and reject malformed input with a clear JSON error rather than a server exception:

- Empty files are rejected immediately.
- A maximum upload size (15MB) is enforced.
- File extensions are cross-checked against the file's actual binary signature (“magic bytes”) so a mislabeled or malicious file cannot masquerade as a supported type by extension alone.
- Corrupted or unreadable PDFs and images are caught explicitly and return a descriptive 400 error instead of propagating a raw exception.

4.3 Semantic Role Resolution

Rather than forcing users into a rigid list, HireLens supports two input modes for the target role: selecting from a curated dropdown of 115+ roles spanning software, business, healthcare, engineering, trades, hospitality, and academia; or typing a free-text description of the desired role.

Free-text input is resolved using sentence-transformer embeddings (all-MiniLM-L6-v2). The user's text and every candidate role's title-plus-description are encoded into vector space, and the closest match is selected via cosine similarity. Role embeddings are computed once at application startup and cached in memory, avoiding redundant computation on every request.

A confidence threshold (cosine similarity ≥ 0.35) determines whether a match is flagged as reliable. Below this threshold, the frontend informs the user that no strong match was found and suggests using the dropdown instead — preventing the system from silently misrepresenting a poor match as a confident one.

4.4 Pakistani-Context, Date-Aware Rating Engine

The rating logic is driven by a system prompt that positions the LLM as an experienced HR recruiter at a mid-to-large Pakistani company, informed by direct research into local hiring conventions, including:

- University and degree recognition patterns specific to the Pakistani education system, with explicit tiering guidance (e.g. NUST, FAST, LUMS, GIKI, IBA vs. solid regional institutes vs. unranked institutions)
- CGPA thresholds commonly used in local screening
- The prevalence and impact of volume-based CV triage on platforms such as Rozee.pk
- Common local CV mistakes: generic filler language, missing quantified impact, unprofessional email addresses, absence of GitHub/LinkedIn links for technical roles

Each rating request includes the current date, and the prompt explicitly instructs the model to treat that date — not its own internal sense of “current year” — as the only valid reference point for judging whether an experience entry is past, current, or (genuinely) future-dated. This was added after testing surfaced false positives where a candidate's current-month internship was incorrectly flagged as a fabricated future date.

The prompt also defines explicit scoring calibration anchors (what a 1–2, 5–6, and 9–10 should each represent to a hiring manager) so category scores are consistently differentiated rather than clustering in a narrow “safe” range, along with edge-case guidance for thin-experience fresh-graduate CVs and CVs with genuinely no red flags.

4.5 Six-Category Structured Evaluation

Every CV is scored across six categories, each returned with a numeric score (1–10) and a specific written comment:

- Format — structure, length, and readability
- University — institution recognition and academic signal
- Experience — relevance and depth of work and project history
- Skills — alignment of technical/professional skills with the target role
- Clarity — quality and precision of written communication
- Red Flags — gaps, inconsistencies, or other concerns a recruiter would notice

4.6 Weighted Overall Scoring

Rather than trusting the LLM's own holistic score — which can be inconsistent — the final overall score is computed deterministically as a weighted average of the six category scores. This produces an explainable, reproducible headline number rather than an opaque model guess.

Category	Weight
Skills	28%
Experience	28%
University	14%
Red Flags	14%
Clarity	10%
Format	6%

This weighting reflects real hiring priorities: skills and experience together account for 56% of the final score, while surface-level formatting contributes only 6% — matching how an experienced recruiter would actually prioritize a CV review.

4.7 Robust AI Response Handling

Gemini is instructed to return only a raw JSON object, but LLM output is never trusted blindly. The backend strips markdown code fences if present, and falls back to a regex-based extraction of the outermost JSON object if the model adds any stray preamble or commentary. If the response still cannot be parsed, or is missing the expected structure, the API returns a clear 502 error rather than crashing or silently returning malformed data to the frontend.

4.8 Actionable Suggestions

Beyond the numeric scores, every rating includes a concise, prioritized list of the top concrete fixes the candidate should make — surfaced clearly in the results dashboard rather than buried in prose.

5. Backend Implementation Detail

5.1 API Endpoints

GET /	Health check — confirms the API is running
POST /extract-text	Accepts a CV file and returns raw extracted text (used for debugging/testing extraction independently of the rating pipeline)
POST /rate-cv	Accepts a CV file and a target role; returns the full structured rating response
GET /roles	Returns the full list of predefined role titles for populating the frontend dropdown

All four endpoints, request parameters, and response shapes are unchanged from the original design — all hardening work was done without breaking the existing frontend contract.

5.2 CORS Configuration

Cross-Origin Resource Sharing is configured via an environment variable (CORS_ALLOW_ORIGINS, comma-separated) rather than a hardcoded value, defaulting to localhost:5173 for local development. This allows the same backend image to serve both local development and the live production frontend (hire-lens-lime.vercel.app) by setting the appropriate origins as a Hugging Face Space secret, without a code change between environments.

5.3 Cross-Platform OCR Toolchain Resolution

Tesseract and Poppler binary paths are resolved automatically and identically across environments, removing the earlier hardcoded Windows-only paths that caused OCR to fail with a `TesseractNotFoundError` on the Linux-based Hugging Face deployment:

- An optional `TESSERACT_CMD` / `POPPLER_PATH` environment variable takes precedence if set, on any platform.
- Otherwise, the binary is located automatically via the system `PATH` (`shutil.which`), which resolves correctly on Linux where `tesseract-ocr` and `poppler-utils` are installed as `apt` packages.
- On Windows only, a conventional local install path is used as a final fallback if the binary isn't found on `PATH`.
- If no binary can be located by any method, a warning is logged at startup rather than failing silently at request time.

5.4 Containerized Deployment

The backend is packaged as a Docker image for deployment on Hugging Face Spaces. The Dockerfile is based on `python:3.13-slim`, installs `tesseract-ocr` and `poppler-utils` as Linux system dependencies via `apt`, runs as a non-root user for security, sets `PYTHONUNBUFFERED` so application logs stream to the Space's log viewer in real time, and exposes the application on port 7860 as required by the Hugging Face Spaces runtime.

5.5 Logging and Observability

Structured logging was added throughout the request lifecycle to make production issues diagnosable directly from the Hugging Face Space's log viewer, without needing to reproduce issues locally:

- Uploaded filename and file size on every request
- Whether a PDF fell back from direct text extraction to OCR, and why
- Gemini request initiation, model used, and CV character count sent
- Gemini success (with computed overall score) or failure (with the underlying error, e.g. rate limiting)
- JSON parsing failures, including the raw model response for debugging

The Gemini API key itself is never written to logs under any code path.

6. Deployment

6.1 Platform Selection

Several free hosting options were evaluated for the backend before settling on a final choice, given the memory footprint of the machine-learning dependencies (`torch`, `transformers`, `sentence-transformers`):

- Render — free tier ruled out due to RAM constraints under the ML dependency load.
- Google Cloud for Students / Oracle Cloud / Fly.io — ruled out due to mandatory credit card requirements.
- Hugging Face Spaces (Docker SDK) — selected. No card required, 16GB RAM on the free CPU tier, well suited to torch-based workloads.

6.2 Live Deployment

Component	Status
Backend API (Hugging Face Spaces, Docker)	Live
Frontend (Vercel)	Live
Gemini LLM Integration	Working
OCR Pipeline (cross-platform)	Working
Upload Validation Layer	Working

Frontend ↔ Backend Connectivity	Working
End-to-End Flow	Verified

6.3 Deployment Workflow

Backend updates:

- Modify code in HireLens/backend
- Copy updated files into the hirelens_backend deployment repository
- Commit and push to the Hugging Face Space's Git remote
- Hugging Face automatically rebuilds the Docker container

Frontend updates:

- Push changes to the main GitHub repository
- Vercel automatically detects the push and redeploys

7. Challenges and Solutions

Local environment setup on Windows

Tesseract OCR and Poppler both required separate binary installation outside of pip, plus correct PATH configuration — resolved by installing each explicitly and verifying via direct command-line version checks before integrating into Python code.

Virtual environment / file placement confusion

Project files were briefly created inside the venv/ folder rather than at the project root due to VS Code's active file context, which caused the running server to silently use a stale file. Resolved by explicitly relocating files and verifying structure via directory listing before proceeding.

Invisible dropdown and input text

Browser-level automatic dark-mode styling of native form elements caused select and input text to render invisibly despite explicit CSS colors. Resolved by adding color-scheme: light to the stylesheet root, forcing light-mode rendering of native form controls regardless of OS/browser theme.

Git tracking gaps during initial push

A stray .gitignore file inside the backend directory (containing a blanket * pattern, left over from virtual environment scaffolding) silently excluded all backend source files from the initial commit. Diagnosed via git status and directory inspection, then resolved by removing the erroneous file and creating a correct top-level .gitignore.

Docker build failure on Hugging Face Spaces

The initial Dockerfile used python:3.11-slim, which was incompatible with a numpy version pinned in requirements.txt that required Python 3.12 or later. Resolved by updating the base image to python:3.13-slim to match the dependency requirements.

Missing LLM API key at runtime

The deployed container initially crashed on startup with an authentication error because the API key was present locally in a .env file (correctly excluded from version control) but not yet configured in the deployment environment. Resolved by adding the provider's API key as a Hugging Face Space secret.

Windows-only OCR paths breaking the Linux deployment

The original code hardcoded a Windows Tesseract executable path and a Windows Poppler bin directory, which worked in local development but caused every image and scanned-PDF upload to fail with a `TesseractNotFoundError` once deployed to the Linux-based Hugging Face container. Resolved by replacing the hardcoded paths with automatic PATH-based detection, with environment-variable overrides and a Windows-only fallback, so the identical codebase runs correctly on both platforms without modification.

Search-grounded rating calls hitting rate limits

An attempt to enable Gemini's built-in Google Search grounding tool — so the model could verify unfamiliar universities and employers in real time — caused 429 `RESOURCE_EXHAUSTED` errors after only a couple of test requests. Investigation showed grounding draws from a separate, more restrictive quota than plain generation and can trigger multiple internal request round-trips per call. Given the added cost, latency, and reliability risk for a use case that doesn't strictly require live web lookups, the feature was reverted in favor of the plain, non-grounded generation call.

False positive: current-month dates flagged as fabricated

Early testing surfaced a rating that penalized a candidate for a July 2026 internship start date as suspicious “future-dated experience,” despite that date being the actual current month. The root cause was that the model has no built-in awareness of the real current date and was judging “future” against its own stale training cutoff. Resolved by explicitly passing today's date into every rating prompt and instructing the model to treat it as the sole authority on past vs. future.

8. Future Improvements

- Rules-based pre-screening layer (CV length, presence of contact links, university-tier lookup table) alongside the LLM evaluation, for faster and more deterministic checks
- Batch CV processing and ranking dashboard for reviewing multiple candidates against a single role at once
- Downloadable PDF report of the rating results
- User accounts and rating history
- Validation study comparing the tool's ratings against real Pakistani HR practitioners' assessments of the same CVs
- Mobile responsiveness pass and further UI polish
- Analytics dashboard for aggregate usage patterns
- Re-evaluate search-grounded verification on a paid Gemini tier with a dedicated, monitored quota, if live university/employer verification becomes a priority

9. Summary

HireLens is a complete, deployed, full-stack AI application that combines large language model reasoning, sentence-embedding-based semantic search, OCR-based document processing, and a deterministic weighted-scoring layer into a single coherent product — one built specifically around the realities of Pakistani corporate hiring rather than adapted from generic Western tooling. Since the initial release, the backend has been hardened with cross-platform OCR resolution, stricter upload validation, more resilient AI response parsing, and a date-aware rating prompt — all without changing the API contract the live frontend depends on. The system is live, publicly accessible, and demonstrates end-to-end ownership of a project from local research and prototyping through to production deployment and post-launch hardening on free-tier infrastructure.

End of document.